Bywaf Usage Guide

Overview

Bywaf is a Python 3 commandlet framework for authorized web application and network testing workflows. It presents a Metasploit-like interactive shell, loads commandlets from plugins, and connects commandlets through a SQLite-backed event bus.

The core idea is simple: one commandlet discovers something and publishes it as an event; another commandlet consumes that event and publishes the next result. For example, `hostscanner` can publish live hosts, `portscanner` can consume those hosts and publish open ports, and HTTP commandlets can consume open ports and probe web services.

Use Bywaf only on systems and networks where you have explicit authorization.

Installation

During development, run Bywaf from the repository root:

```
cd bywaf
python3 -m bywaf --help
python3 -m bywaf repl
```

For an editable local install:

```
python3 -m pip install -e .
bywaf --help
bywaf repl
```

The project metadata defines a console script named `bywaf`, so packaged installations can expose Bywaf as a normal command instead of requiring `python3 -m bywaf`.

The host and port scanner commandlets use `nmap` through a Python adapter. A local `nmap` binary is required for real scans. The adapter prefers `nmaplib`, then `python-nmap`, then `nmapthon`, then `libnmap`.

Encrypted databases require SQLCipher support. On Debian or Ubuntu, install the SQLCipher library and use the optional Python extra:

```
sudo apt install sqlcipher libsqlcipher-dev
python3 -m pip install -e '.[sqlcipher]'
```

Starting Bywaf

Start the interactive shell:

```
bywaf
```

or, from a source checkout:

```
python3 -m bywaf
```

Create or open the default database with SQLCipher encryption:

```
bywaf --encrypted
bywaf --database client.sqlite3 --encrypted
```

Run one command non-interactively:

```
bywaf run ls
bywaf run cat README.md
bywaf run 'hostscanner 127.0.0.1 | portscanner'
```

Simple `run` commands do not need quotes. Use quotes when the command contains shell metacharacters such as `|`, `&`, `>`, or spaces that must be preserved inside a single argument.

REPL Basics

The REPL prompt is:

```
bywaf>
```

Commandlets can be run directly:

```
bywaf> ls
bywaf> cat README.md
bywaf> hostscanner 127.0.0.1
```

Built-in commands manage the shell and the event database:

```
help
help <command>
plugins
cmds
vars
history
jobs
runs
topics
db <status|path|checkpoint|vacuum|new|encrypt|decrypt|rekey>
show <topic|job=id|run=id|pipeline=id>
load <resource>
save <resource>
exit
```

`help <command>` shows the same help as `<command> --help` for commandlets.

Commandlets

A commandlet is an executable unit exposed by a plugin. Each commandlet declares its name, description, arguments, consumed topics, and emitted topics.

Examples:

```
bywaf> help hostscanner
bywaf> hostscanner --help
bywaf> portscanner --help
```

Many scanning commandlets support `-s` or `--silent` to suppress console alerts. Without silent mode, commandlets print discovery alerts such as:

```
hostscanner <hostscanner-...>: discovered host 127.0.0.1
portscanner <portscanner-...>: discovered port 127.0.0.1:80/tcp
```

Commandlets can also declare tab-completion behavior for their arguments and options. For example, `ls [path]`, `cat <path>`, and `less <path>` get filename completion because those commandlets declare path/file completion in their plugin specs. Other completion specs include `topic`, `run`, `pipeline`, `job`, and `plugin`, so plugin authors can make hand-typed commands much easier to complete correctly.

Plugins

A plugin provider groups related commandlets. The `plugins` command lists loaded providers:

```
bywaf> plugins
discovery
http
network
os
storage
```

The `cmds` command lists commandlets grouped by provider:

```
bywaf> cmds
discovery
  hostscanner
http
  http_headers
  http_probe
network
  portscanner
os
  cat
  less
```

```
  ls
storage
  db
```

Bundled plugins are listed in `bywaf/plugins/plugins.json`. Adding a plugin file is not enough to load it by default; add its dotted path to that config.

Load an additional plugin by name from `.bywaf/plugins`:

```
bywaf> load plugin=myplugin
```

Load a plugin from an explicit filesystem path:

```
bywaf> load plugin=./plugins/myplugin
bywaf> load plugin=~/bywaf-plugins/myplugin
```

Pipelines

Pipelines connect commandlets with `|`:

```
bywaf> hostscanner 127.0.0.1 | portscanner
```

The runner executes each stage in order. Events emitted by one stage are passed to the next stage as input. Events are also stored in SQLite with a pipeline ID and command run ID.

This model allows downstream commandlets to consume only the output relevant to the current pipeline, rather than every historical event in the database.

Background Execution

Append `&` to background a commandlet or pipeline:

```
bywaf> hostscanner 192.168.0.1-255 &
```

Stage-level backgrounding works inside pipelines:

```
bywaf> hostscanner 192.168.0.1-255 & | portscanner &
```

In that example, `portscanner` listens for `host.found` rows created by the immediately upstream `hostscanner` run in the same pipeline. It does not consume unrelated `host.found` rows from older scans.

List jobs:

```
bywaf> jobs
```

Show one job:

```
bywaf> show job=<id>
```

Database and Event Model

Bywaf stores events in SQLite. The default database is:

```
.bywaf/bywaf.sqlite3
```

SQLite is used in WAL mode. Inserts are committed immediately; there is no separate document-style save step. On shutdown, Bywaf checkpoints the WAL using `PRAGMA wal_checkpoint(TRUNCATE)` so WAL contents are folded back into the main database file.

List known event topics:

```
bywaf> topics
```

Show recent events for a topic:

```
bywaf> show host.found
bywaf> show port.open
```

List command runs:

```
bywaf> runs
```

Show events by command run or pipeline:

```
bywaf> show run=<command-run-id>
bywaf> show pipeline=<pipeline-id>
```

Save a database snapshot:

```
bywaf> save db=snapshot.sqlite3
```

Save an encrypted snapshot:

```
bywaf> save --encrypt db=snapshot.sqlite3
```

Inspect or maintain the active database:

```
bywaf> db status
bywaf> db path
bywaf> db checkpoint
bywaf> db vacuum
```

Create a fresh database and switch the active session to it:

```
bywaf> db new
bywaf> db new --file=client.sqlite3
bywaf> db new --encrypt --file=client.sqlite3
bywaf> db new --force --file=client.sqlite3
```

Without `--file`, `db new` creates a timestamped database under `.bywaf/db/`. With `--file`, it refuses to overwrite an existing file. Add `--force` to move the existing database and SQLite sidecar files to timestamped `.bak-*` names before creating the new DB. `--encrypt` forces SQLCipher encryption and prompts twice for a passphrase.

The session variable `db.encryption=sqlcipher` makes `db new` encrypted by default:

```
bywaf> vars db.encryption=sqlcipher
bywaf> db new
```

Convert the active database in place:

```
bywaf> db encrypt
bywaf> db rekey
bywaf> db decrypt
```

`db encrypt` converts the active plaintext database to SQLCipher and prompts twice for a new passphrase. `db rekey` changes the passphrase for an encrypted database. `db decrypt` exports the active encrypted database back to plaintext SQLite after an explicit `YES` confirmation.

Switch to another database:

```
bywaf> load db=snapshot.sqlite3
```

If the database is encrypted, Bywaf prompts for its passphrase when loading it. Passphrases are kept in process memory only and are not written to config or history files.

Resource Files

Bywaf keeps default state in:

```
.bywaf/
```

Important files:

```
.bywaf/bywaf.sqlite3
.bywaf/config.json
.bywaf/history.bywaf
.bywaf/plugins/
```

Resource resolution rules are consistent:

```
plugin=<name>   -> .bywaf/plugins/<name>
script=<name>   -> ./<name>
db=<name>       -> ./<name>
config=<name>   -> ./<name>
history=<name>  -> ./<name>
```

Explicit paths are used as filesystem paths for every resource type:

```
./name
../name
~/name
/absolute/name
```

Examples:

```
bywaf> load script=scan.bywaf
bywaf> load script=./scripts/scan.bywaf
bywaf> save config=session.json
bywaf> load config=session.json
bywaf> save history=session-history.bywaf
bywaf> load history=session-history.bywaf
```

Variables

List variables:

```
bywaf> vars
```

Set a variable:

```
bywaf> vars name=value
```

Common examples:

```
bywaf> vars http_probe.cookie-file=/tmp/cookies.txt
bywaf> vars history.timestamp-format=%Y-%m-%d %H:%M:%S %Z
```

Save variables:

```
bywaf> save config=config.json
```

Load variables:

```
bywaf> load config=config.json
```

Config files are JSON objects containing session variables.

History

Every non-empty REPL command is appended to:

```
.bywaf/history.bywaf
```

History lines are stored as commands followed by a timestamp comment:

```
hostscanner 127.0.0.1  # 2026-05-12 10:15:30 EDT
```

The `history` command shows only commands from the current REPL invocation:

```
bywaf> history
```

The persistent history file can be viewed with the OS commandlets:

```
bywaf> cat .bywaf/history.bywaf
bywaf> less .bywaf/history.bywaf
```

Because timestamps are comments, history lines can be copied into a script file.

Change the timestamp format:

```
bywaf> vars history.timestamp-format=%Y/%m/%d %H:%M:%S %Z
```

Scripts

Scripts are text files with one command expression per line:

```
# scan local host
hostscanner 127.0.0.1
portscanner --from-topic host.found
```

Blank lines and lines beginning with `#` are ignored. Inline comments are also allowed after whitespace:

```
plugins  # list loaded plugin providers
```

Run a script:

```
bywaf> load script=scan.bywaf
```

Bundled Commandlets

os

`ls` lists local files:

```
bywaf> ls
bywaf> ls bywaf/plugins
```

`cat` prints a local text file:

```
bywaf> cat README.md
```

`less` opens the system `less` pager when running interactively:

```
bywaf> less README.md
```

Use `/` to search inside `less`, arrow keys or paging keys to scroll, and `q` to quit.

discovery

`hostscanner` discovers live hosts with nmap:

```
bywaf> hostscanner 127.0.0.1
bywaf> hostscanner 192.168.0.1-255
bywaf> hostscanner 192.168.1-3.1-255
```

It emits `host.found` events.

network

`portscanner` scans hosts for open ports:

```
bywaf> portscanner 127.0.0.1
bywaf> portscanner --ports 22,80,443 127.0.0.1
```

If `--ports` is omitted, nmap uses its normal default top-port behavior. It emits `port.open` events.

Use listen mode to consume newly inserted hosts:

```
bywaf> portscanner --listen
```

http

`http_headers` performs HTTP HEAD requests and emits `http.headers` events:

```
bywaf> http_headers example.com
bywaf> http_headers --ssl true example.com
```

`http_probe` probes HTTP endpoints and emits `http.endpoint` events:

```
bywaf> http_probe https://example.com/
```

For authorized session-aware testing, it can use cookies:

```
bywaf> vars http_probe.cookie-file=/path/to/cookies.txt
bywaf> http_probe https://example.com/
bywaf> http_probe --firefox-profile ~/.mozilla/firefox/<profile>
```

Common Workflows

Scan one host for live status:

```
bywaf> hostscanner 127.0.0.1
```

Scan one host for ports:

```
bywaf> portscanner 127.0.0.1
```

Discover hosts and scan their ports:

```
bywaf> hostscanner 192.168.0.1-255 | portscanner
```

Run discovery and port scanning in the background:

```
bywaf> hostscanner 192.168.0.1-255 & | portscanner &
```

Probe HTTP services after port scanning:

```
bywaf> hostscanner 127.0.0.1 | portscanner --ports 80,443 | http_probe
```

Save the current database:

```
bywaf> save db=scan-results.sqlite3
bywaf> save --encrypt db=scan-results.sqlite3
```

Save variables and session history:

```
bywaf> save config=session.json
bywaf> save history=session.bywaf
```

Troubleshooting

If `python` is not available, use `python3`:

```
python3 -m bywaf
```

If scanning fails with an nmap error, verify that `nmap` is installed and that the selected Python nmap binding is available.

If socket creation is denied, the process may be running inside a sandbox or without the privileges needed by the selected scan type.

If a command is unknown, check loaded commandlets:

```
bywaf> cmds
```

If a plugin does not load, verify its directory structure and that it defines a `plugin()` factory returning a commandlet.

SQLite WAL files such as `.sqlite3-wal` and `.sqlite3-shm` are normal. Bywaf checkpoints the WAL on shutdown.

Developer Notes

The main package layout is:

```
bywaf/app.py          REPL and built-in commands
bywaf/runner.py       parsing, pipelines, foreground/background execution
bywaf/db.py           SQLite event store
bywaf/plugin.py       commandlet protocol and specs
bywaf/registry.py     plugin discovery and loading
bywaf/completion.py   readline completion
bywaf/plugins/        bundled plugin providers
tests/                unit tests
```

Run tests:

```
python3 -m unittest discover -s tests
```

Add a commandlet by defining a class with a `CommandSpec` and a `run()` method, then expose it through a `plugin()` factory. Add bundled commandlets to `bywaf/plugins/plugins.json` when they should load by default.

Commandlets can declare completion metadata with `ArgumentSpec`, `OptionSpec(..., completion=CompletionSpec(...))`, or an optional custom `complete(context, args, prefix)` method. See `PLUGIN_AUTHOR_GUIDE.md` for a walkthrough and a small working example.

Reference

Useful built-ins:

```
help [command]
plugins
cmds
vars [name=value]
history
jobs
runs
topics
show <topic>
show job=<id>
show run=<id>
show pipeline=<id>
db <status|path|checkpoint|vacuum|new|encrypt|decrypt|rekey>
load plugin=<resource>
load script=<resource>
load db=<resource>
load config=<resource>
load history=<resource>
save db=<resource>
save --encrypt db=<resource>
save config=<resource>
save history=<resource>
prompt [pattern]
exit
```

Common event topics:

```
host.found
port.open
http.headers
http.endpoint
job.started
job.finished
job.failed
```